Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

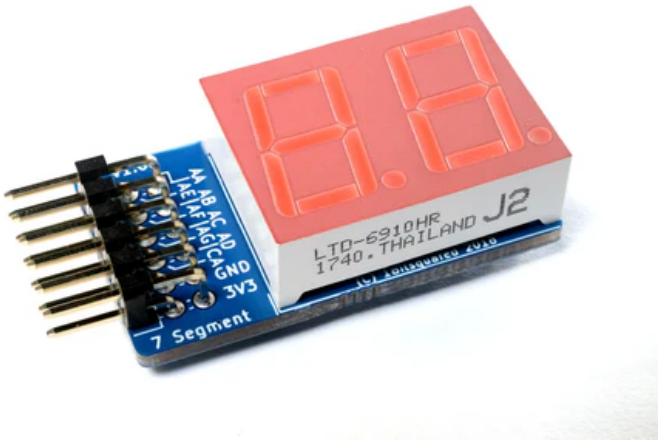
VHDL-Übung 04: Bedingte Zuweisung, If-Anweisung, For-Anweisung

M.Sc. Eike Trumann, M.Sc. Jasper Homann

Prof. Dr.-Ing. Guillermo Payá Vayá

2. Februar 2026

Aufgabe VHDL-400: *7-Segment-Anzeige*



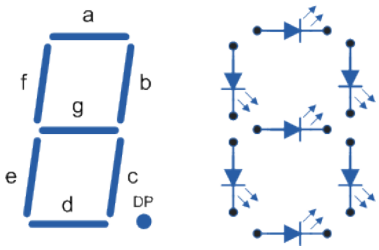
Beim Aufbau von einfachen digitalen Elektroniksystemen werden verbreitet 7-Segment-Anzeigen zur Darstellung von Zahlenwerten verwendet. Diese haben den Vorteil, dass sie sehr leicht in Digitallogik gesteuert werden können.

Für diese Aufgabe nehmen wir ein sogenanntes Common-Cathode-Display an. Hier ist es so, dass jedes Segment aus einer LED besteht, die über einen Ausgang der Digitallogikschaltung so gesteuert werden kann, dass das Segment leuchtet, wenn eine 1 ausgegeben wird und ausgeschaltet ist, wenn eine 0 ausgegeben wird.

VHDL-400

Um dieses betreiben zu können, müssen Ziffern in die entsprechenden Segmente umgewandelt werden.

```
entity segment_decoder is
    port(
        digit_i    : in  std_ulogic_vector (3 downto 0);
        segments_o : out std_ulogic_vector (0 to 6)
    );
end entity segment_decoder;
```



In der Abbildung dargestellt ist die Zuordnung der Bits zu den einzelnen Segmenten. Der Ausgangsvektor des `segment_decoder` ist dabei alphabetisch von a entsprechend 0 bis g entsprechend 6 geordnet.

Implementieren Sie den Decoder.

Aufgabe VHDL-401: *Zweier-Logarithmus*

In dieser Aufgabe soll der Logarithmus zur Basis 2 von einer achtbittigen vorzeichenlosen Zahl berechnet werden. Dabei dürfen neben grundlegenden Logikelementen auch `if`-Anweisungen verwendet werden, aus denen beispielsweise auch eine Look-up-Table erstellt werden darf.

VHDL-401

```
entity log_calculator is
port(
        data_bus     : in  std_ulogic_vector(7 downto 0);
        result       : out std_ulogic_vector(2 downto 0)
);
end log_calculator;
```

Aufgabe VHDL-402: *Noch mehr Schiebeoperationen*

Nachdem wir im letzten Blatt bereits eine einfache Schiebeoperation kennengelernt haben, wollen wir nun eine umfangreichere Verschiebeeinheit erstellen. Für die Umsetzung dieser Aufgabe können `case`-Anweisungen verwendet werden.

Im Vergleich zu letzter Woche haben wir zwei neue Eingänge hinzugefügt: `amount` enthält eine vorzeichenlose Ganzzahl, die angibt, um wie viele Bits die Schiebeoperation durchgeführt werden soll. Diese ist 3 Bit breit, sodass alle sinnvollen Werte zwischen 0 und 7 möglich sind.

Die Richtung des Schifts wird nun durch `left_shift` festgelegt. Ist dieses eine 0, so soll nach rechts geschiftet werden, ist hingegen eine 1 enthalten, so soll nach links geschiftet werden. Bei einem Linksschift braucht das Vorzeichen nicht beachtet zu werden.

VHDL-402

```
entity best_calculator is
port(
        data_bus     : in  std_ulogic_vector(7 downto 0);
        amount       : in  std_ulogic_vector(2 downto 0);
        logic_shift  : in  std_ulogic;
        left_shift   : in  std_ulogic;
        result       : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

Aufgabe VHDL-403: *Mustererkennung*

Ein häufiger Anwendungsfall von Hardwarebeschleunigern ist die Erkennung von Mustern. Für diesen Anwendungsfall ist die Verwendung des "for" Schlüsselwortes in VHDL geeignet.

Es soll erkannt werden, wie häufig das drei Bit lange Muster (beispielsweise "101") in einer längeren Bitfolge (beispielsweise "1001010010111 01") vorkommt. Dabei sollten alle Vorkommen des Musters gezählt werden, auch solche, die sich überlappen (beispielsweise "10101" sollte als 2 Vorkommen des Musters gezählt werden). Der Counter soll dann in Form einer vorzeichenlosen Ganzzahl ausgeben, wie häufig das Muster gefunden wurde.

VHDL-403

```
entity pattern_matcher is
port(
  data_bus     : in  std_ulogic_vector(255 downto 0);
  pattern      : in  std_ulogic_vector(2 downto 0);
  counter      : out std_ulogic_vector(7 downto 0)
);
end pattern_matcher;
```

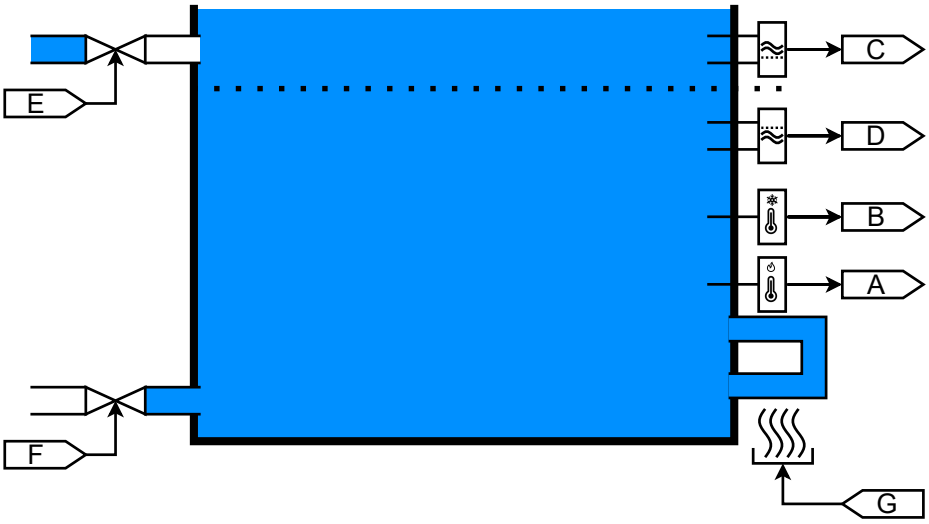
Aufgabe VHDL-404: *Schwimmbecken*

Damit ein Schwimmbecken gute Bedingungen zum Schwimmen bietet, ist es erforderlich, dass der Wasserstand und die Wassertemperatur ständig reguliert werden.

Ein einfaches Schwimmbecken wurde zu diesem Zweck mit folgenden Systemen ausgestattet:

VHDL-404

- (A) Ein Sensor für überhöhte Temperaturen, dieser gibt bei zu hoher Temperatur 1 aus, ansonsten 0
- (B) Ein Sensor für zu niedrige Temperaturen, dieser gibt bei zu niedriger Temperatur 1 aus, ansonsten 0
- (C) Ein Wasserstandssensor am oberen Rand, dieser gibt 1 aus, wenn der Wasserstand den oberen Rand erreicht, 0 wenn er (wie im Sollzustand) im trockenen liegt.
- (D) Ein Wasserstandssensor knapp unter dem Sollstand. Dieser gibt ebenfalls 1 aus, wenn dieser im Wasser liegt, was hier dem Sollzustand entspricht. Gibt dieser Sensor eine 0 aus, ist der Wasserstand zu niedrig.
- (E) Ein regelbares Zulaufventil, mit dem kaltes Wasser in das Becken zugeführt werden kann.
- (F) Ein regelbares Ablaufventil, mit dem Wasser aus dem Becken abgelassen werden kann.
- (G) Eine regelbare Heizung, die das Wasser im Schwimmbecken erwärmen kann.



Die Äußere Schnittstelle des Reglers wird in VHDL folgendermaßen definiert:

```
entity pool_regulation is
port(
  high_temp_sensor_i    : in  std_ulogic;
  low_temp_sensor_i     : in  std_ulogic;
  high_level_sensor_i   : in  std_ulogic;
  low_level_sensor_i    : in  std_ulogic;
  cold_water_inlet_o    : out std_ulogic;
  water_drain_o         : out std_ulogic;
  heater_o              : out std_ulogic;
);
end pool_regulation;
```

Es sollen nun folgende Regeln gelten:

- Bei korrekter Temperatur und zu hohem Wasserstand wird Wasser abgelassen (`water_drain_o = 1`).
- Bei korrekter Temperatur und niedrigem Wasserstand wird Wasser zugeführt (`cold_water_inlet_o = 1`) und gleichzeitig geheizt (`heater_o = 1`).
- Ist die Temperatur zu hoch, wird kaltes Wasser zugeführt. Nur, wenn das maximale Wasserniveau erreicht ist, wird gleichzeitig Wasser abgelassen.
- Alle weiteren Regeln sollen aus der Problemstellung abgeleitet werden.

Implementieren die Poolsteuerung. Da einige Eingangszustände unmöglich sind, können Sie eine Logikminimierung durchführen und hierbei auch dont-care-Werte verwenden.

Aufgabe VHDL-405: *Fertigungsüberwachung*

Ein Messsensor für die Formabweichung von in einer Fabrik gefertigten Kunststoffteilen soll verwendet werden, um die Fertigungslinie anzuhalten, falls eine Abweichung außerhalb der akzeptablen Toleranz auftritt.

VHDL-405

Das Messergebnis stellt die Abweichung in Form einer vier Bit großen, vorzeichenbehafteten Zweierkomplementzahl **`messwert_i`** dar. Der Wertebereich der tatsächlichen Messergebnisse liegt ausschließlich zwischen -6 und 6 (diese können vorkommen), größere und kleinere Werte können nicht vorkommen.

Die Produktion soll angehalten werden, wenn der Betrag der Formabweichung vom Sollwert 0 echt größer als 3 ist. Dafür wird ein Ausgang namens **`ok_o`** verwendet, der eine 1 ausgeben soll, solange die Maschine weiter laufen darf.

Wenn ein Fehler behoben wurde, wird die Maschine manuell wieder in Betrieb genommen, ohne, dass der Ausgang `ok_o` beachtet wird. Der Messwert kann sich entsprechend auch ändern, während die Maschine aus Sicht der Qualitätsprüfung gestoppt sein sollte.

```
entity kunststoff_form_check is
port(
  messwert_i : in  std_ulogic_vector(3 downto 0);
  ok_o       : out std_ulogic;
);
end kunststoff_form_check;
```

Aufgabe VHDL-406: *Pumpspeicherkraftwerk*

Es soll eine Steuerung für ein Pumpspeicherkraftwerk entwickelt. Diese soll rein kombinatorisch aus den Eingangssignalen ein Steuersignal erzeugen.

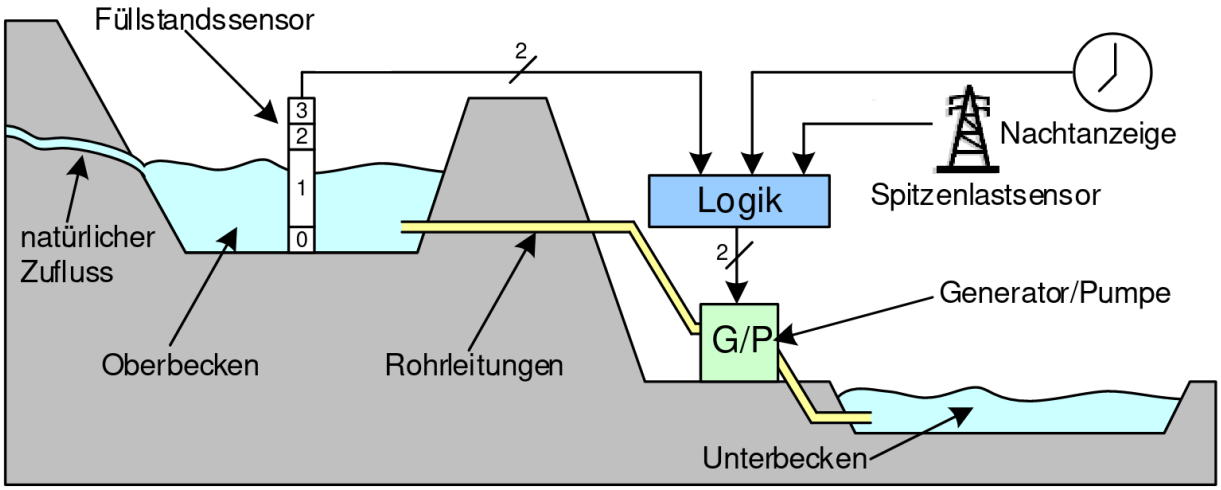


Abbildung 1: Schematische Zeichnung des Pumpspeicherkraftwerkes

Falls die obere Füllstandsgrenze des Oberbeckens noch nicht erreicht ist, wird nachts (`night_i` = 1) Wasser durch Rohrleitungen in das Oberbecken gepumpt (`control_o` = 10). Ein Sensor (2-Bit-Signal `level_i`) misst den Füllstand des Oberbeckens. Bedingt durch einen natürlichen Zufluss kann die obere Füllstandsgrenze überschritten werden. In diesem Fall muss Wasser abgelassen werden (`control_o` = 01) bis die obere Füllstandsgrenze wieder erreicht ist. Das Ablassen von Wasser (`control_o` = 01) zur Stromerzeugung ist nur dann erforderlich, wenn Spitzenlasten (`peak_load_i` = 1) auftreten. Wasser darf aber nur abgelassen werden, falls das Oberbecken nicht leer ist.

VHDL-406

Tabelle 1: Werte des Füllstandssensors am Oberbecken:

Signal	Wert	Bedeutung
`level_i`	00	leer
`level_i`	01	normal
`level_i`	10	obere Füllstandsgrenze
`level_i`	11	übervoll (obere Grenze überschritten)

Tabelle 2: Werte des Steuerungsausganges:

Signal	Wert	Bedeutung
`control_o`	00	Stillstand
`control_o`	01	Wasser ablassen (Generator)
`control_o`	10	Oberbecken auffüllen (Pumpe)
`control_o`	11	verboten

```
entity pumpspeicher is
port(
```

```
  level_i     : in  std_ulogic_vector(1 downto 0);
  night_i     : in  std_ulogic;
  peak_load_i : in  std_ulogic;
  control_o   : out std_ulogic_vector(1 downto 0)
);
end entity;
```

Implementieren Sie die vorgegebene Steuerungslogik ausschließlich auf Basis von grundlegenden Logikgattern.

Aufgabe VHDL-407: *Laserpistole zur Verkehrsüberwachung*

Zur Durchführung von mobilen Geschwindigkeitskontrollen setzt die Polizei Laserpistolen ein. Um die Arbeit zu erleichtern wird eine hardwarebasierte automatische Berechnung der Fahrverbotsdauer benötigt.

Diese erhält eine 4 bit Eingabe x. Das höchstwertige bit kodiert ob die Geschwindigkeitsmessung innerorts ($x_3 = 0$) oder außerorts (x_3) durchgeführt wurde. Die restlichen drei bit ($x_2x_1x_0 = 0$) kodieren die gemessene Überschreitung der Höchstgeschwindigkeit in km/h.

Die Ausgabe ist die Dauer des Fahrverbots in Monaten als vorzeichenlose 2-bit Binärzahl.

Die Überschreitung und das vom Messort abhängige resultierende Fahrverbot sind in der folgenden Tabelle dargestellt:

VHDL-407

$x_2x_1x_0$	Überschreitung	Fahrverbot (y_1y_0) innerorts	Fahrverbot (y_1y_0) außerorts
000	< 20	0	0
001	21-30	0	0
010	31-40	1	0
011	41-50	1	1
100	51-60	2	1
101	61-70	3	2
110	> 70	3	3

```
entity laserpistole is
  port(
        x :  in std_ulogic_vector(3 downto 0);
        y : out std_ulogic_vector(1 downto 0)
  );
end entity laserpistole;
```

Implementieren Sie die vorgegebene die Logik der Laserpistole. Verwenden Sie don't cares für die ungenutzte Eingangskombinationen.

Aufgabe VHDL-408: *Binärisierung von Graustufen*

Bei der automatisierten Bildanalyse werden verschiedene Vorverarbeitungsschritte eingesetzt. Einer dieser Schritte ist die Binärisierung.

VHDL-408

Bei der Binarisierung wird aus einem Graustufen-Bild ein Schwarz-Weiß-Bild erzeugt, indem jeder Pixel mit zwei Schwellenwerten t_{low} und t_{high} verglichen wird. Nur, wenn für den Wert des Pixels p $t_{\text{low}} <= p <= t_{\text{high}}$ gilt, wird Schwarz (0) ausgegeben, ansonsten wird Weiß (1) ausgegeben. Für diese Aufgabe soll $t_{\text{low}} = 6$ und $t_{\text{high}} = 9$ angenommen werden. Um die Binarisierung zu Beschleunigen, soll eine Schaltung erstellt werden, die die Binarisierung eines 64x64 großen Bildes parallel berechnen kann.

(a) Graustufen-Bild mit 4-bit pro Pixel

(b) Binärisierung mit 1-bit pro Pixel

Die Eingabe ist ein 64x64 großes 2D Array von vorzeichenlosen 4-bit Binärzahlen, wobei jedes Element des Arrays den Wert eines Eingabepixels kodiert. Die Ausgabe ist ein 64x64 großes 2D Array von 1-bit breiten Werten, wobei jedes Element die Binärisierung des entsprechenden Eingabepixels darstellt.

```
entity binarisierung is
  port(
        image_i : in  image_4bit_t;
        image_o : out image_1bit_t
  );
end entity;
```

Die beiden zweidimensionalen Array Typen `image_4bit_t` und `image_1bit_t` sind in folgendem Package definiert:

```
package package_408 is
        type image_4bit_t is array (0 to 63, 0 to 63) of std_ulogic_vector(3 downto 0);
        type image_1bit_t is array (0 to 63, 0 to 63) of std_ulogic;
end package;
```

Die einzelnen Werte können jeweils mit `image(x)(y)` adressiert werden, wobei `x` und `y` integer (z. B. die Zählvariablen einer for-Schleife) sind.

Implementieren Sie die parallele Binärisierung des Eingabebildes mithilfe von eines Prozesses und for-Schleifen.

Aufgabe VHDL-409: *Roboter*

In dieser Aufgabe soll die Wanderkennung eines einfachen fahrenden Roboters implementiert werden.

VHDL-409

Dieser verfügt über zwei Drucksensoren, die in Fahrtrichtung vorne links (x_3x_2) und vorne rechts (x_1x_0) angebracht sind. Von diesen erhält die Wanderkennung den gemessenen Druck jeweils als eine 2-Bit Vorzeichenlose Binärzahl. Aufgrund der Konstruktion des Roboters können jedoch nie beide Sensoren gleichzeitig den maximalen Wert (11) messen.

Die Wanderkennung gibt aus, ob und wo der Roboter eine Wand berührt. Dabei gibt es vier Möglichkeiten: Keine Wand (y_1y_0 = 00), Wand links (y_1y_0 = 10), Wand rechts (y_1y_0 = 01), Wand voraus (y_1y_0 = 11).

Der Algorithmus zur Wandererkennung funktioniert folgendermaßen:

- Wenn beide Sensoren den gleichen Wert ausgeben, wird entweder keine (beide 0) oder sowohl links als auch rechts (beide > 0) eine Wand erkannt.
- Meldet ein Sensor einen Wert von 1 und der andere eine 0 wird (noch) keine Wand erkannt.
- In den anderen restlichen (möglichen) Fällen wird eine Wand auf der Seite erkannt, auf der der Sensor mit dem größeren Messwert ist.

```
entity roboter is
  port(
        x : in  std_ulogic_vector(3 downto 0);
        y : out std_ulogic_vector(1 downto 0)
  );
end entity;
```

Implementieren Sie die Wanderkennung. Verwenden Sie don't cares für unmögliche Eingaben.

Aufgabe VHDL-410: *Abstandsregelung*

Es soll ein Fahrassistenzsystem zur automatischen Abstandsregelung (Adaptive Cruise Control, ACC) implementiert werden.

VHDL-410

Diese erhält als Eingabe die aktuelle Geschwindigkeit (`speed_i`) und den von einem Radar gemessenen Abstand zum vorausfahrenden Fahrzeug (`distance_i`) jeweils als 2-bit vorzeichenlose Binärzahl. Je größer die Geschwindigkeit bzw. der Abstand, desto höher ist der gemessene Wert.

Die Ausgabe der Abstandsregelung ist eine Anweisung (`control_o`) an die Geschwindigkeitsregelung. Die möglichen Anweisungen sind: Bremsen (00), Beschleunigen (01), Stark bremsen (10), Geschwindigkeit halten (11).

Die Anweisung hängt von der Differenz der Geschwindigkeit (G) des Abstands (A) ab:

Differenz	Anweisung
$G - A = 0$	Geschwindigkeit halten
$G - A = 1$	Bremsen
$G - A > 1$	Stark bremsen
$G - A < 0$	Beschleunigen

```
entity acc is
  port(
        speed_i    : in  std_ulogic_vector(1 downto 0);
        distance_i : in  std_ulogic_vector(1 downto 0);
        control_o  : out std_ulogic_vector(1 downto 0)
  );
end entity;
```

Implementieren Sie die Logik der Abstandsregelung.

Aufgabe VHDL-411: *One-hot Komperator*

Es soll eine Komperatorschaltung für zwei vorzeichenlose 2-bit Binärzahlen a und b implementiert werden.

Die Ausgabe erfolgt mit einer sogenannten one-hot Kodierung. Dabei ist immer genau eines der möglichen Ausgabebits aktiv.

Der Komperator hat drei Ausgangsbits:

- x, für $a = b$
- y, für $a < b$
- z, für $a > b$

VHDL-411

```
entity logik is
  port(
        a : in  std_ulogic_vector(1 downto 0);
        b : in  std_ulogic_vector(1 downto 0);
        x : out std_ulogic;
        y : out std_ulogic;
        z : out std_ulogic
  );
end entity;
```

Implementieren Sie die Logik des one-hot Komperators.